claude-windows / 902e27a6-e026-4b55-8c26-f2c47329cc44

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\902e27a6-e026-4b55-8c26-f2c47329cc44\subagents\workflows\wf_c287c071-8e9\agent-a1503123b7babe194.jsonl`
- SHA-256: `cd7ea3eadc091f365153984c203c203c798524732e12896726cc38d77767b1e2`
- Source modified: `2026-07-07T14:48:50+00:00`
- Imported at: `2026-07-08T16:00:01+00:00`
- project: `wf_c287c071-8e9`
- session_id: `902e27a6-e026-4b55-8c26-f2c47329cc44`

Transcript

1. user (2026-07-07T14:45:11.303Z)

```
You are reviewing an uncommitted change in a git worktree at:
  C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval
Branch: fix/worker-skip-revalidation-on-dispatch (based on origin/master).

THE BUG BEING FIXED: On the cloud-serving path, the control-plane (backend/orchestration.py)
runs
validators and ONLY dispatches to the Cloud Run worker on PASS. The worker
(backend/worker/__main__.py
cmd_infer) then REDUNDANTLY re-ran validation with its OWN config (different
min_blur_threshold),
rejecting a clip the control-plane already passed. Live failure 2026-07-07: passed at
min_blur_threshold=8.0 on the control-plane, worker rejected at its default 20.0 ? GPU dispatch
succeeded but the CUJ failed with worker rc=2.

THE FIX (skip-on-dispatch): a new ComputeJobSpec.skip_validation bool (default False) ?
backend/compute/cloud_run.py emits a "--skip-validation" worker CLI flag when set ?
backend/worker/__main__.py cmd_infer honors it (skips validator_registry.run_all_with_context,
sets val_results=[], val_context={}) ? backend/orchestration.py::_maybe_dispatch_remote sets
skip_validation=True on every dispatch. The direct "worker infer" CLI (no control-plane) still
validates by default. New tests in tests/test_worker.py and tests/test_api_cloud_dispatch.py.

Inspect the ACTUAL code ? do NOT trust this summary. Run:  cd
"C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval" && git --no-pager diff
and read the surrounding functions (not just the diff hunks). The full test suite already passes
(1779 passed), ruff + mypy are clean, coverage 85.42%. So do NOT report style nits or things the
gate already covers. Hunt for REAL correctness/rollout/consumer defects the tests would miss.
Only report a finding you can tie to a concrete failure scenario in THIS repo's code.

LENS: TEST ADEQUACY & REGRESSION SAFETY. Read the two new tests
(tests/test_worker.py::test_cmd_infer_skip_validation_bypasses_validators,
test_parser_infer_skip_validation_flag; tests/test_api_cloud_dispatch.py::
test_cloud_dispatch_skips_worker_revalidation) and the functions they exercise.
- Do the tests actually PROVE the fix, or could they pass while the bug persists? (e.g. does the
  dispatch test prove the flag reaches the worker argv AND that the worker would honor it end-
to-end?)
- Is the acceptance criterion met: "a video that passes the control-plane gate is NOT rejected
by the
  worker on the cloud path"? Is there an end-to-end-ish test that a low-sharpness clip survives
the
  cloud path now? If the coverage is only unit-level, is that a real gap?
- Are the negative/default-off paths covered (direct CLI still validates; skip_validation
defaults
  False; _dispatch_remote propagation)?
```

Report only material test gaps that would let a real regression ship undetected.

2. assistant (2026-07-07T14:45:15.066Z)

I'll inspect the actual change and the tests. Let me start by reading the diff and the relevant files.

3. user (2026-07-07T14:45:17.338Z)

```
diff --git a/backend/compute/base.py b/backend/compute/base.py
index 279606e..31af3fe 100644
--- a/backend/compute/base.py
+++ b/backend/compute/base.py
@@ -28,6 +28,13 @@ class ComputeJobSpec:
     segmenter: Optional[str] = None
     # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
frozen/hashable.
     config_overrides: tuple[str, ...] = ()
+    # Skip the worker's redundant re-validation. The DISPATCHER (the cloud-serving control-
plane) is
+    # the authoritative validation gate: it runs the SAME validators with ITS resolved config
and only
+    # dispatches on PASS, so a worker re-run is (a) redundant, (b) a second full-video decode,
and
+    # (c) outright WRONG when the worker's config resolves different thresholds ? it can REJECT
a clip
+    # the gate already passed. Default False keeps the direct ``worker infer`` CLI (no control-
plane)
+    # validating exactly as before.
+    skip_validation: bool = False


 @dataclass(frozen=True)
diff --git a/backend/compute/cloud_run.py b/backend/compute/cloud_run.py
index 841785d..7247ace 100644
--- a/backend/compute/cloud_run.py
+++ b/backend/compute/cloud_run.py
@@ -196,6 +196,10 @@ class CloudRunCompute(ComputeBackend):
         ]
         if spec.segmenter:
             argv += ["--segmenter", spec.segmenter]
+        if spec.skip_validation:
+            # The control-plane already validated + only dispatches on PASS ? tell the worker
not to
+            # re-decode and re-validate (redundant, and its config may resolve different
thresholds).
+            argv.append("--skip-validation")
         for kv in (*spec.config_overrides, *self._container_overrides):
             argv += ["--set", kv]

diff --git a/backend/orchestration.py b/backend/orchestration.py
index 5e9f510..79b4fed 100644
--- a/backend/orchestration.py
+++ b/backend/orchestration.py
@@ -568,11 +568,18 @@ def _maybe_dispatch_remote(
         prefetch = getattr(wasb_cfg, "prefetch_batches", None)
         if prefetch:
             overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
+    # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
authoritative
+    # gate ? it already ran the validators with its resolved config just above (only a PASS
reaches
+    # here; a failure returned "failed" before dispatch). A worker re-run would decode the
video a
+    # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
```

```
thresholds and
+    # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
+    # then worker-rejected at its default 20.0 after a successful GPU dispatch).
     spec = ComputeJobSpec(
         video_uri=req.video_path,
         out_uri=out_root,
         segmenter=seg_name,
         config_overrides=tuple(overrides),
+        skip_validation=True,
     )
     try:
         # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
index 88c50f6..9b19215 100644
--- a/backend/worker/__main__.py
+++ b/backend/worker/__main__.py
@@ -186,6 +186,7 @@ def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
         out_uri=args.out_uri,
         segmenter=args.segmenter,
         config_overrides=tuple(args.set or ()),
+        skip_validation=bool(getattr(args, "skip_validation", False)),
     )
     try:
         result = compute.run_infer(spec)
@@ -259,10 +260,25 @@ def cmd_infer(args: argparse.Namespace) -> int:
     # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be byte-
identical).
     video_id = compute_video_id(local_video)

-    # 3. VALIDATE (same registry as the main CLI).
-    val_results, val_context = validator_registry.run_all_with_context(
-        local_video, config
-    )
+    # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
+    # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate: it runs
the
+    # SAME validators with ITS resolved config and only dispatches on PASS. Re-validating here
is
+    # redundant, decodes the whole video a SECOND time, and is outright WRONG when the worker's
config
+    # resolves different thresholds than the control-plane's ? observed 2026-07-07: a 5312x2988
clip
+    # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0, then the
worker
+    # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the fact.
+    # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote) skips
it; the
+    # direct ``worker infer`` CLI has no control-plane, so it still validates by default.
+    if getattr(args, "skip_validation", False):
+        print(
+            "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
+            "gate and already validated this input."
+        )
+        val_results, val_context = [], {}
+    else:
+        val_results, val_context = validator_registry.run_all_with_context(
+            local_video, config
+        )
     failures = [r for r in val_results if not r.passed]
     db = Database(os.path.join(scratch, config.output.db_name))
     db.add_video(
@@ -724,6 +740,14 @@ def build_parser() -> argparse.ArgumentParser:
         metavar="k=v",
```

```
         help="config override, e.g. --set indexing.skip_ai_handoff=true",
     )
+    pi.add_argument(
+        "--skip-validation",
+        action="store_true",
+        help="skip the worker's re-validation: the dispatcher (control-plane) already ran the "
+        "validators with its resolved config and only dispatches on PASS, so re-validating here
is "
+        "redundant, decodes the video again, and can contradict the gate. Default-OFF: the
direct "
+        "`worker infer` CLI still validates.",
+    )
     pi.set_defaults(func=cmd_infer)

     pt = sub.add_parser(
diff --git a/tests/test_api_cloud_dispatch.py b/tests/test_api_cloud_dispatch.py
index 8382234..19c37d9 100644
--- a/tests/test_api_cloud_dispatch.py
+++ b/tests/test_api_cloud_dispatch.py
@@ -236,6 +236,22 @@ def test_cloud_dispatch_omits_unset_wasb_tuning(cloud_env):
     assert "indexing.wasb.prefetch_batches" not in joined


+def test_cloud_dispatch_skips_worker_revalidation(cloud_env):
+    """The control-plane is the AUTHORITATIVE validation gate: it validated + only dispatches
on PASS
+    (registry.run_all_with_context is stubbed to pass in cloud_env). Re-validating on the
worker is
+    redundant, decodes the video a second time, and ? because the control-plane's validation
config
+    isn't forwarded ? can REJECT a clip the gate already passed (observed 2026-07-07: passed at
+    validation.min_blur_threshold=8.0, then worker-rejected at its default 20.0). So the
dispatch MUST
+    carry --skip-validation, telling the worker to trust the gate. Regression guard for that
wiring."""
+    _db, _bucket, monkeypatch = cloud_env
+    fake = _FakeCloudClient()
+    _inject_fake(monkeypatch, fake)
+    m._execute_processing(
+        m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
+    )
+    assert "--skip-validation" in fake.calls[0]
+
+
 def test_cloud_failure_marks_failed_no_rows(cloud_env):
     db, _bucket, monkeypatch = cloud_env
     # pre-seed a prior good row ? must be preserved on a cloud failure (destroy-AFTER-confirm).
diff --git a/tests/test_worker.py b/tests/test_worker.py
index 8fb5e16..72b2514 100644
--- a/tests/test_worker.py
+++ b/tests/test_worker.py
@@ -97,6 +97,14 @@ def test_parser_infer_accepts_set_and_uris():
     assert args.set == ["indexing.skip_ai_handoff=true", "sport=tennis"]


+def test_parser_infer_skip_validation_flag():
+    base = ["infer", "--video-uri", "s3://b/v.mp4", "--out-uri", "s3://b"]
+    # Default-OFF: the direct `worker infer` CLI still validates.
+    assert wmain.build_parser().parse_args(base).skip_validation is False
+    # The dispatcher passes it to disable the redundant worker re-validation.
+    assert wmain.build_parser().parse_args(base + ["--skip-validation"]).skip_validation is
True
+
+
 # ---------------------------------------------------------------- cmd_infer (local, mocked)
```

```
@@ -407,6 +415,43 @@ def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
     assert marker["returncode"] == 2 and marker["status"] == "failed"


+def test_cmd_infer_skip_validation_bypasses_validators(tmp_path, monkeypatch):
+    """--skip-validation (set by the dispatcher, the authoritative gate) must NOT run the
worker's
+    validators ? no second decode, and no chance of a threshold mismatch REJECTING a clip the
+    control-plane already passed (the live 2026-07-07 CUJ failure). Prove it by making
+    run_all_with_context RAISE: a skipped run never touches it, so the job completes rc 0 with
a
+    status="success" report. Contrast test_cmd_infer_validation_failure_returns_2 (no flag ? rc
2)."""
+    monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidSkip")
+
+    def _boom(*a, **k):  # the validators must never be reached on the skip path
+        raise AssertionError("run_all_with_context must not run under --skip-validation")
+
+    monkeypatch.setattr(wmain.validator_registry, "run_all_with_context", _boom)
+    monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FakeSeg)
+    monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
+    video = tmp_path / "in.mp4"
+    video.write_bytes(b"FAKE")
+    out = tmp_path / "bucket"
+
+    rc = wmain.main(
+        [
+            "infer",
+            "--video-uri",
+            str(video),
+            "--out-uri",
+            str(out),
+            "--scratch",
+            str(tmp_path / "s"),
+            "--segmenter",
+            "wasb_hybrid",
+            "--skip-validation",
+        ]
+    )
+    assert rc == 0  # a clip that WOULD fail re-validation still succeeds ? the gate is trusted
+    report = json.loads((out / "outputs" / "vidSkip" / "report.json").read_text())
+    assert report["status"] == "success" and report["segment_count"] == 2
+
+
 # --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push bundle ---
```

4. assistant (2026-07-07T14:45:22.372Z)

Let me read the surrounding functions and the test infrastructure in parallel.

5. user (2026-07-07T14:45:24.260Z)

```
1       """API cloud-serving dispatch (COMPUTE_DECOUPLED_SERVING M6 ? the API path).
2
3       Proves the /api/process job path can offload DETECT to a Cloud Run GPU Job and ingest
the rally
4       intervals back into the API DB, so /api/query + /api/compile (local stitch) are
unchanged ? all
5       GPU-free + cloud-free via a FAKE CloudRunJobClient that writes report.json +
intervals.csv + the
6       done-marker into a local "bucket". Cardinal invariant: DEFAULT-OFF ? only
compute_target=cloud_run
7       dispatches; every other target runs the in-process segmenter byte-identically.
8       """
9
```

```
    import csv
    import json
    import os

    import pytest

    pytest.importorskip("httpx")
    from fastapi.testclient import TestClient

    import backend.compute as compute_pkg
    import backend.main as m
    from backend.compute.cloud_run import CloudRunJobClient
    from backend.config.models import AppConfig
    from backend.infrastructure.database import Database
    from backend.infrastructure.execution_policy import ExecutionPolicy
    from backend.pipeline.validators.base import ValidationResult

    _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)


    def _pass():
        return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]


    class _InlineExecutor:
        def submit(self, fn, *a, **k):
            fn(*a, **k)


    class _FakeCloudClient(CloudRunJobClient):
        """Simulates the Cloud Run worker: on dispatch, write report.json + intervals.csv +
the done-marker
        into the local bucket exactly where the real worker would, so the bucket-poll +
ingest resolve."""

        def __init__(self, *, video_id="vidCloud", returncode=0, rows=None):
            self.calls: list[list[str]] = []
            self.video_id, self.returncode = video_id, returncode
            self.rows = (
                rows
                if rows is not None
                else [(2.74, 6.91, 5, "winner"), (8.74, 11.38, 3, "unforced_error")]
            )

        def run_job(self, job_name, argv, *, region, project=None):
            self.calls.append(list(argv))
            out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
            done_uri = argv[argv.index("--done-uri") + 1]
            if self.returncode == 0:
                outputs = os.path.join(out_uri, "outputs", self.video_id)
                os.makedirs(outputs, exist_ok=True)
                with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
                    json.dump(
                        {
                            "video_id": self.video_id,
                            "segment_count": len(self.rows),
                            "status": "success",
                        },
                        f,
                    )
                with open(
                    os.path.join(outputs, "intervals.csv"),
                    "w",
                    newline="",
                    encoding="utf-8",
```

```
                ) as f:
                    w = csv.writer(f)
                    w.writerow(["start", "end", "shot_count", "ending_reason"])
                    for s, e, sc, er in self.rows:
                        w.writerow([f"{s:.3f}", f"{e:.3f}", sc, er])
            os.makedirs(os.path.dirname(done_uri), exist_ok=True)
            with open(done_uri, "w", encoding="utf-8") as f:
                json.dump(
                    {
                        "video_id": self.video_id,
                        "returncode": self.returncode,
                        "segment_count": len(self.rows),
                        "status": "success",
                    },
                    f,
                )
            return "exec-1"

        def cancel_execution(self, execution):
            raise AssertionError(
                "cancel must never fire on the happy path"
            )  # pragma: no cover


    @pytest.fixture
    def cloud_env(tmp_path, monkeypatch):
        """API wired for cloud_run with a LOCAL bucket (the local storage backend makes the
bucket read an
        identity file read ? no GCS). A gs:// input is 'fetched' to a local fake file for
hash+validate."""
        db = Database(str(tmp_path / "t.db"))  # noqa: F841
        monkeypatch.setattr(m, "db_instance", db)
        monkeypatch.setattr(m, "db_instance", db)
        bucket = tmp_path / "bucket"
        bucket.mkdir()
        cfg = AppConfig.model_validate(
            {  # noqa: F841
                "deployment": {"compute_target": "cloud_run"},
                "storage": {"backend": "local", "output_root": str(bucket)},
            }
        )
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
        # A real gs:// input would be fetched to hash+validate; return a local fake so that
path is offline.
        fake_local = tmp_path / "fetched.mp4"
        fake_local.write_bytes(b"FAKEVIDEOBYTES")
        monkeypatch.setattr(
            m.storage, "acquire_local_input", lambda path, scfg: str(fake_local)
        )
        monkeypatch.setattr(
            m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
        )
        return db, bucket, monkeypatch


    def _inject_fake(monkeypatch, fake):
        """Make backend.compute.get_compute_backend (re-imported inside
_maybe_dispatch_remote) build a
        real CloudRunCompute around the FAKE client + the fast no-wait policy + a fixed
token. ``**kw``
        forwards the per-request ``target_override`` (item 3) so the cloud-picker path
resolves too."""
        real = compute_pkg.get_compute_backend
```

```
        monkeypatch.setattr(
            compute_pkg,
            "get_compute_backend",
            lambda config, **kw: real(
                config, run_client=fake, policy=_FAST, token="tok", **kw
            ),
        )


    def _meta(row):
        md = row.get("metadata")
        return json.loads(md) if isinstance(md, str) else (md or {})


    def test_cloud_dispatch_ingests_segments(cloud_env):
        db, _bucket, monkeypatch = cloud_env
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)

        out = m._execute_processing(
            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
        )

        assert out["status"] == "success"
        assert len(out["play_segments"]) == 2
        rows = db.query_play_segments(out["video_id"], {})
        assert len(rows) == 2
        metas = [_meta(r) for r in rows]
        assert all(md["source"] == "cloud_run" for md in metas)
        assert {md.get("shot_count") for md in metas} == {5, 3}
        assert {md.get("ending_reason") for md in metas} == {"winner", "unforced_error"}


    def test_cloud_dispatch_builds_correct_spec(cloud_env):
        _db, bucket, monkeypatch = cloud_env
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)

        m._execute_processing(
            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
        )

        assert len(fake.calls) == 1  # exactly one Cloud Run Job execution
        argv = fake.calls[0]
        assert argv[argv.index("--video-uri") + 1] == "gs://b/clip.mp4"
        assert argv[argv.index("--out-uri") + 1] == str(bucket)
        assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
        joined = " ".join(argv)
        assert (
            "deployment.compute_target=local_gpu" in joined
        )  # container runs INLINE (never re-dispatch)
        assert "indexing.detector_impl=native" in joined
        assert (
            "indexing.skip_ai_handoff=false" in joined
        )  # the engine's AI-handoff stance is forwarded


    def test_cloud_dispatch_forwards_skip_ai_handoff_true(cloud_env):
        """When the engine runs no-AI (skip_ai_handoff=True), the cloud job must too ? else
it tries the
        Gemini handoff (which the cloud job has no key for) and yields 0 rallies. The flag
is forwarded."""
        _db, _bucket, monkeypatch = cloud_env
        m.global_config.indexing.skip_ai_handoff = True
        fake = _FakeCloudClient()
```

```
        _inject_fake(monkeypatch, fake)
        m._execute_processing(
            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
        )
        assert "indexing.skip_ai_handoff=true" in " ".join(fake.calls[0])


    def test_cloud_dispatch_forwards_wasb_serving_knobs(cloud_env):
        """The worker Job has no DEPLOYMENT_PROFILE, so it never applies the cloud-serving
preset ?
        the control-plane MUST forward its resolved WASB serving knobs (decode_backend +
GPU-feed
        tuning) or the worker silently reverts to the model defaults (cpu decode, batch 8),
making the
        preset flip (#506/#507) a no-op on the real dispatch path. Regression guard for that
wiring."""
        _db, _bucket, monkeypatch = cloud_env
        m.global_config.indexing.wasb.decode_backend = "nvdec_resident"
        m.global_config.indexing.wasb.batch_size = 64
        m.global_config.indexing.wasb.prefetch_batches = 4
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
        m._execute_processing(
            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
        )
        joined = " ".join(fake.calls[0])
        assert "indexing.wasb.decode_backend=nvdec_resident" in joined
        assert "indexing.wasb.batch_size=64" in joined
        assert "indexing.wasb.prefetch_batches=4" in joined


    def test_cloud_dispatch_omits_unset_wasb_tuning(cloud_env):
        """decode_backend defaults to cpu (always forwarded ? harmless, keeps worker ==
control-plane),
        but batch_size/prefetch_batches default to None and must NOT be forwarded (None =
the worker's
        parity default; forwarding an empty value would reach argparse as a bad token)."""
        _db, _bucket, monkeypatch = cloud_env
        # defaults: decode_backend="cpu", batch_size=None, prefetch_batches=None
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
        m._execute_processing(
            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
        )
        joined = " ".join(fake.calls[0])
        assert "indexing.wasb.decode_backend=cpu" in joined
        assert "indexing.wasb.batch_size" not in joined
        assert "indexing.wasb.prefetch_batches" not in joined


    def test_cloud_dispatch_skips_worker_revalidation(cloud_env):
        """The control-plane is the AUTHORITATIVE validation gate: it validated + only
dispatches on PASS
        (registry.run_all_with_context is stubbed to pass in cloud_env). Re-validating on
the worker is
        redundant, decodes the video a second time, and ? because the control-plane's
validation config
        isn't forwarded ? can REJECT a clip the gate already passed (observed 2026-07-07:
passed at
        validation.min_blur_threshold=8.0, then worker-rejected at its default 20.0). So the
dispatch MUST
        carry --skip-validation, telling the worker to trust the gate. Regression guard for
that wiring."""
        _db, _bucket, monkeypatch = cloud_env
        fake = _FakeCloudClient()
```

```
        _inject_fake(monkeypatch, fake)
        m._execute_processing(
            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
        )
        assert "--skip-validation" in fake.calls[0]


    def test_cloud_failure_marks_failed_no_rows(cloud_env):
        db, _bucket, monkeypatch = cloud_env
        # pre-seed a prior good row ? must be preserved on a cloud failure (destroy-AFTER-
confirm).
        db.add_video("vidLocalPrior", "gs://b/clip.mp4", "processed", [])
        fake = _FakeCloudClient(returncode=1)
        _inject_fake(monkeypatch, fake)

        out = m._execute_processing(
            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
        )
        assert out["status"] == "failed"
        assert "returncode 1" in out["message"]
        assert out["play_segments"] == []
        assert db.query_play_segments(out["video_id"], {}) == []
        # It DID dispatch + run remotely (then failed rc=1), so the path is honestly
cloud_run + dispatched.
        assert (
            out["compute"]["path"] == "cloud_run" and out["compute"]["dispatched"] is True
        )


    def test_cloud_requires_a_bucket_input(cloud_env, tmp_path):
        _db, _bucket, monkeypatch = cloud_env
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
        local_path = str(tmp_path / "local_only.mp4")

        out = m._execute_processing(
            m.ProcessRequest(video_path=local_path, segmenter="wasb_hybrid")
        )
        assert out["status"] == "failed"
        assert "cloud input" in out["message"]
        assert fake.calls == []  # never dispatched a local-only path


    def test_cloud_stages_local_upload_to_bucket(cloud_env, tmp_path, monkeypatch):
        """A browser upload lands on THIS box's local disk; under cloud-serving with a CLOUD
input_root it
        is STAGED to the bucket and the L4 job dispatches from the staged gs:// URI (not
hard-failed) ? #461.
        The DB video record is repointed at the staged source so a later /api/compile can
re-fetch it."""
        db, _bucket, _mp = cloud_env
        m.global_config.storage.input_backend = "gcs"
        m.global_config.storage.input_root = "gs://in-bucket/videos"
        puts: list = []

        def _fake_put(src, uri, **kw):
            puts.append((str(src), uri, kw.get("config")))
            return uri

        monkeypatch.setattr(m.storage, "put", _fake_put)
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
        local_path = str(tmp_path / "upload.mp4")

        out = m._execute_processing(
```

```
            m.ProcessRequest(video_path=local_path, segmenter="wasb_hybrid")
        )

        assert out["status"] == "success"
        # 1) the local upload was staged UP to the input bucket, namespaced by video_id
        assert len(puts) == 1
        src, dest, put_cfg = puts[0]
        assert src == local_path
        assert dest.startswith("gs://in-bucket/videos/") and dest.endswith("/upload.mp4")
        # the active storage settings are threaded into put() (so a non-default S3 endpoint
/ GCS project /
        # etc. survives) ? not a fresh default client. The model_dump carries
input_root/input_backend.
        assert put_cfg is not None and put_cfg.get("input_root") == "gs://in-bucket/videos"
        assert put_cfg.get("input_backend") == "gcs"
        # 2) the L4 job dispatched from the STAGED gs:// URI, not the unreadable local path
        assert len(fake.calls) == 1
        argv = fake.calls[0]
        assert argv[argv.index("--video-uri") + 1] == dest
        # 3) the DB video record now points at the staged source (so compile re-fetches it)
        row = db.get_video(out["video_id"])
        assert row is not None and row["filepath"] == dest


    def test_cloud_stages_when_input_root_is_cloud_and_input_backend_default(
        cloud_env, tmp_path, monkeypatch
    ):
        """input_root='gs://?' with input_backend left at its DEFAULT ('local') is a valid
cloud config ?
        the storage layer classifies an explicit root by its scheme (resolve_input_uri joins
under
        input_root before ever consulting input_backend), so the staging gate must too (PR
#467 review).
        Regression: the gate previously required input_backend != 'local' and hard-failed
this shape."""
        _db, _bucket, _mp = cloud_env
        m.global_config.storage.input_root = "gs://in-bucket/videos"
        # input_backend deliberately NOT set ? stays at the model default 'local'
        assert m.global_config.storage.input_backend == "local"
        puts: list = []

        def _fake_put(src, uri, **kw):
            puts.append((str(src), uri))
            return uri

        monkeypatch.setattr(m.storage, "put", _fake_put)
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
        local_path = str(tmp_path / "upload.mp4")

        out = m._execute_processing(
            m.ProcessRequest(video_path=local_path, segmenter="wasb_hybrid")
        )

        assert out["status"] == "success"  # NOT the 'needs a cloud input' pre-dispatch
failure
        assert len(puts) == 1
        src, dest = puts[0]
        assert src == local_path
        assert dest.startswith("gs://in-bucket/videos/") and dest.endswith("/upload.mp4")
        # dispatch continued from the STAGED gs:// URI
        assert len(fake.calls) == 1
        argv = fake.calls[0]
        assert argv[argv.index("--video-uri") + 1] == dest

```

```

    def test_cloud_local_upload_without_input_root_still_fails(cloud_env, tmp_path,
monkeypatch):
        """No cloud input_root configured ? we genuinely can't stage a local upload ? hard-
fail (no
        dispatch), rather than dispatching a path the Cloud Run job can't read."""
        _db, _bucket, _mp = cloud_env  # fixture leaves input_root unset
(input_backend=local)
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
        called = []
        monkeypatch.setattr(m.storage, "put", lambda *a, **k: called.append(a))

        out = m._execute_processing(
            m.ProcessRequest(video_path=str(tmp_path / "up.mp4"), segmenter="wasb_hybrid")
        )
        assert out["status"] == "failed"
        assert "cloud input" in out["message"]
        assert fake.calls == [] and called == []  # neither staged nor dispatched


    def test_default_off_runs_in_process(tmp_path, monkeypatch):
        """compute_target='' ? get_compute_backend returns None ? the in-process segmenter
runs
        byte-identically (the gate is inert). No Cloud Run dispatch."""
        db = Database(str(tmp_path / "t.db"))  # noqa: F841
        monkeypatch.setattr(m, "db_instance", db)
        monkeypatch.setattr(m, "db_instance", db)
        cfg = AppConfig()  # compute_target defaults to ""  # noqa: F841
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(
            m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
        )
        video = tmp_path / "v.mp4"
        video.write_bytes(b"hashable-bytes")

        class _InProcSeg:
            def __init__(self, db_, cfg_):
                self.db = db_

            def process_video(self, vid, path, *a, **k):
                sid = self.db.add_play_segment(vid, 0.0, 5.0)
                return [
                    {
                        "id": sid,
                        "start_time": 0.0,
                        "end_time": 5.0,
                        "duration": 5.0,
                        "metadata": {},
                    }
                ]

        monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
        out = m._execute_processing(
            m.ProcessRequest(video_path=str(video), segmenter="motion")
        )
        assert out["status"] == "success"
        assert len(out["play_segments"]) == 1
        assert "cloud_run" not in out["message"]


    def test_cloud_ingest_failure_marks_failed_no_leak(cloud_env):
        """A rc=0 marker but a MISSING intervals.csv (a partial push) must fail cleanly +
```

```
leak no rows
        (destroy-AFTER-confirm), not 500 or hang."""
        db, _bucket, monkeypatch = cloud_env

        class _NoCsvClient(_FakeCloudClient):
            def run_job(self, job_name, argv, *, region, project=None):
                self.calls.append(list(argv))
                out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
                done_uri = argv[argv.index("--done-uri") + 1]
                outputs = os.path.join(out_uri, "outputs", self.video_id)
                os.makedirs(outputs, exist_ok=True)
                with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
                    json.dump({"video_id": self.video_id, "status": "success"}, f)
                # deliberately DO NOT write intervals.csv
                os.makedirs(os.path.dirname(done_uri), exist_ok=True)
                with open(done_uri, "w", encoding="utf-8") as f:
                    json.dump({"video_id": self.video_id, "returncode": 0}, f)
                return "exec"

        _inject_fake(monkeypatch, _NoCsvClient())
        out = m._execute_processing(
            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
        )
        assert out["status"] == "failed"
        assert "could not read the result" in out["message"]
        assert db.query_play_segments(out["video_id"], {}) == []


    def test_cloud_video_id_divergence_is_tolerated_with_warning(cloud_env, caplog):
        """If the worker re-hashes different bytes (result.video_id != local id), ingest
still proceeds
        under the LOCAL id but logs a WARNING so the orphaned-report condition is
diagnosable."""
        db, _bucket, monkeypatch = cloud_env
        _inject_fake(monkeypatch, _FakeCloudClient(video_id="DIVERGED"))
        with caplog.at_level("WARNING", logger="backend.main"):
            out = m._execute_processing(
                m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
            )
        assert out["status"] == "success"
        assert len(db.query_play_segments(out["video_id"], {})) == 2
        assert any(
            "DIVERGED" in r.getMessage() and "!= local" in r.getMessage()
            for r in caplog.records
        )


    class _InProcSeg:
        """Minimal in-process segmenter (writes one play row) ? proves the local path
actually ran."""

        def __init__(self, db_, cfg_):
            self.db = db_

        def process_video(self, vid, path, *a, **k):
            sid = self.db.add_play_segment(vid, 0.0, 5.0)
            return [
                {
                    "id": sid,
                    "start_time": 0.0,
                    "end_time": 5.0,
                    "duration": 5.0,
                    "metadata": {},
                }
            ]
```

```
488
489
490     # --- item 3: SPA compute-picker (per-request compute override + cloud_available
capability) ---
491
492
493     def test_per_request_local_forces_in_process(cloud_env):
494         """Server configured for cloud (compute_target=cloud_run), but a per-request
compute='local'
495         forces the in-process segmenter and NEVER dispatches to Cloud Run."""
496         db, _bucket, monkeypatch = cloud_env
497         fake = _FakeCloudClient()
498         _inject_fake(monkeypatch, fake)
499         monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
500
501         out = m._execute_processing(
502             m.global_config,
503             m.db_instance,
504             m.ProcessRequest(
505                 video_path="gs://b/clip.mp4", segmenter="motion", compute="local"
506             ),
507         )
508         assert out["status"] == "success"
509         assert len(out["play_segments"]) == 1
510         assert "cloud_run" not in out["message"]
511         assert (
512             fake.calls == []
513         )  # explicit "run on my machine" ? no dispatch despite a cloud server
514
515
516     def test_per_request_cloud_forces_dispatch_on_local_server(tmp_path, monkeypatch):
517         """Server default is LOCAL (compute_target=''), but a per-request compute='cloud'
forces a Cloud
518         Run dispatch when the control plane is wired (CLOUD_RUN_JOB + GOOGLE_CLOUD_PROJECT +
a bucket)."""
519         db = Database(str(tmp_path / "t.db"))  # noqa: F841
520         monkeypatch.setattr(m, "db_instance", db)
521         bucket = tmp_path / "bucket"
522         bucket.mkdir()
523         cfg = AppConfig.model_validate(
524             {  # noqa: F841
525                 "deployment": {"compute_target": ""},  # server default = in-process
526                 "storage": {"backend": "local", "output_root": str(bucket)},
527             }
528         )
529         monkeypatch.setattr(m, "global_config", cfg)
530         monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
531         fake_local = tmp_path / "fetched.mp4"
532         fake_local.write_bytes(b"FAKEVIDEOBYTES")
533         monkeypatch.setattr(
534             m.storage, "acquire_local_input", lambda path, scfg: str(fake_local)
535         )
536         monkeypatch.setattr(
537             m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
538         )
539         monkeypatch.setenv("CLOUD_RUN_JOB", "rally-worker")
540         monkeypatch.setenv("GOOGLE_CLOUD_PROJECT", "proj")
541         fake = _FakeCloudClient()
542         _inject_fake(monkeypatch, fake)
543
544         out = m._execute_processing(
545             m.global_config,
546             m.db_instance,
547             m.ProcessRequest(
548                 video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="cloud"
```

```
            ),
        )
        assert out["status"] == "success"
        assert len(out["play_segments"]) == 2
        assert (
            len(fake.calls) == 1
        )  # per-request override dispatched despite the LOCAL server default


    def test_per_request_cloud_unconfigured_fails_clearly(tmp_path, monkeypatch):
        """compute='cloud' on a server NOT wired for it fails clearly (no silent local
fallback, no
        dispatch) so the picker can surface 'cloud isn't available here'."""
        db = Database(str(tmp_path / "t.db"))  # noqa: F841
        monkeypatch.setattr(m, "db_instance", db)
        cfg = AppConfig.model_validate(
            {"deployment": {"compute_target": ""}}
        )  # no bucket, no env  # noqa: F841
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(
            m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
        )
        monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
        monkeypatch.delenv("GOOGLE_CLOUD_PROJECT", raising=False)
        video = tmp_path / "v.mp4"
        video.write_bytes(b"hashable-bytes")

        out = m._execute_processing(
            m.global_config,
            m.db_instance,
            m.ProcessRequest(
                video_path=str(video), segmenter="wasb_hybrid", compute="cloud"
            ),
        )
        assert out["status"] == "failed"
        assert "isn't configured" in out["message"]


    def test_unknown_compute_value_falls_back_to_server_default(cloud_env, caplog):
        """An unrecognised compute value is ignored (logged) and the server default applies
? here a
        cloud_run server still dispatches to the cloud."""
        db, _bucket, monkeypatch = cloud_env
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
        with caplog.at_level("WARNING", logger="backend.main"):
            out = m._execute_processing(
                m.global_config,
                m.db_instance,
                m.ProcessRequest(
                    video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="banana"
                ),
            )
        assert out["status"] == "success"
        assert len(fake.calls) == 1  # fell back to the configured cloud_run default
        assert any("unknown compute" in r.getMessage() for r in caplog.records)


    def test_capabilities_reports_cloud_available(cloud_env):
        """A cloud_run-configured server advertises deployment.cloud_available=True (the SPA
shows the
        cloud toggle only then)."""
        _db, _bucket, _monkeypatch = cloud_env
        app = m.create_app(m.global_config, m.db_instance)
        client = TestClient(app)
```

```
611         resp = client.get("/api/capabilities")
612         caps = resp.json()
613         assert caps["deployment"]["cloud_available"] is True
614         assert caps["deployment"]["compute_target"] == "cloud_run"
615
616
617     def test_capabilities_cloud_unavailable_by_default(monkeypatch, tmp_path):
618         """A default (local) server with no Cloud Run env advertises
cloud_available=False."""
619         cfg = AppConfig()
620         db = Database(str(tmp_path / "t.db"))
621         app = m.create_app(cfg, db)
622         monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
623         monkeypatch.delenv("GOOGLE_CLOUD_PROJECT", raising=False)
624         client = TestClient(app)
625         resp = client.get("/api/capabilities")
626         caps = resp.json()
627         assert caps["deployment"]["cloud_available"] is False
628
629
630     def test_full_api_process_to_query_via_cloud(cloud_env):
631         """End-to-end through the public API: POST /api/process (gs:// input) ? the inline
worker dispatches
632         to (fake) Cloud Run ? job done ? POST /api/query returns the ingested segments."""
633         db, _bucket, monkeypatch = cloud_env
634         fake = _FakeCloudClient(video_id="vidCloud2")
635         _inject_fake(monkeypatch, fake)
636         app = m.create_app(m.global_config, db)
637         client = TestClient(app)
638
639         r = client.post(
640             "/api/process",
641             json={"video_path": "gs://b/clip.mp4", "segmenter": "wasb_hybrid"},
642         )
643         assert r.status_code == 202
644         job_id = r.json()["job_id"]
645
646         job = client.get(f"/api/jobs/{job_id}").json()
647         assert job["status"] == "done"
648         video_id = job["video_id"]
649         assert video_id
650
651         q = client.post("/api/query", json={"video_id": video_id})
652         assert q.status_code == 200
653         segs = q.json()["play_segments"]
654         assert len(segs) == 2
655         assert all(_meta(s)["source"] == "cloud_run" for s in segs)
656
657
658     def test_local_override_survives_api_boundary(cloud_env):
659         """REGRESSION: the picker choice must survive the async submit?worker?reconstruct
boundary. On a
660         cloud_run server, POST /api/process with compute='local' must persist (jobs.compute)
+ reconstruct
661         (_job_to_request) the choice and run IN-PROCESS ? not dispatch to Cloud Run. (An
earlier cut
662         dropped compute in create_job/_job_to_request, silently nullifying the picker
through /api/process.)"""
663         db, _bucket, monkeypatch = cloud_env
664         fake = _FakeCloudClient()
665         _inject_fake(monkeypatch, fake)
666         monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
667         app = m.create_app(m.global_config, db)
668         client = TestClient(app)
669
```

```
        r = client.post(
            "/api/process",
            json={
                "video_path": "gs://b/clip.mp4",
                "segmenter": "motion",
                "compute": "local",
            },
        )
        assert r.status_code == 202
        job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
        assert job["status"] == "done"
        assert (
            fake.calls == []
        )  # 'local' honoured across the boundary ? never dispatched to Cloud Run
        segs = db.query_play_segments(job["video_id"], {})
        assert len(segs) == 1
        assert all(
            _meta(s).get("source") != "cloud_run" for s in segs
        )  # in-process rows, not cloud


    def test_cloud_override_survives_api_boundary_on_local_server(tmp_path, monkeypatch):
        """REGRESSION (mirror): on a LOCAL-default server wired for cloud (env + bucket),
POST /api/process
        with compute='cloud' must survive the boundary and dispatch to Cloud Run."""
        db = Database(str(tmp_path / "t.db"))  # noqa: F841
        monkeypatch.setattr(m, "db_instance", db)
        bucket = tmp_path / "bucket"
        bucket.mkdir()
        cfg = AppConfig.model_validate(
            {  # noqa: F841
                "deployment": {"compute_target": ""},  # server default = in-process
                "storage": {"backend": "local", "output_root": str(bucket)},
            }
        )
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
        fake_local = tmp_path / "fetched.mp4"
        fake_local.write_bytes(b"FAKEVIDEOBYTES")
        monkeypatch.setattr(
            m.storage, "acquire_local_input", lambda path, scfg: str(fake_local)
        )
        monkeypatch.setattr(
            m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
        )
        monkeypatch.setenv("CLOUD_RUN_JOB", "rally-worker")
        monkeypatch.setenv("GOOGLE_CLOUD_PROJECT", "proj")
        fake = _FakeCloudClient(video_id="vidBoundary")
        _inject_fake(monkeypatch, fake)
        app = m.create_app(m.global_config, db)
        client = TestClient(app)

        r = client.post(
            "/api/process",
            json={
                "video_path": "gs://b/clip.mp4",
                "segmenter": "wasb_hybrid",
                "compute": "cloud",
            },
        )
        assert r.status_code == 202
        job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
        assert job["status"] == "done"
        assert (
            len(fake.calls) == 1
```

```
        )  # 'cloud' honoured across the boundary ? dispatched despite local default
        segs = db.query_play_segments(job["video_id"], {})
        assert len(segs) == 2
        assert all(_meta(s)["source"] == "cloud_run" for s in segs)


    # --- compute-path PROVENANCE (the validation method: prove which backend ran)
------------------


    def test_cloud_dispatch_stamps_compute_provenance(cloud_env):
        """A cloud run stamps result['compute'] with path=cloud_run + the REAL Cloud Run
execution name
        (the GCP-verifiable proof) + the requested choice + job/region/outputs ? so the
intended path is
        provable from the job result alone (no guessing from segment metadata)."""
        _db, _bucket, monkeypatch = cloud_env
        _inject_fake(monkeypatch, _FakeCloudClient())

        out = m._execute_processing(
            m.ProcessRequest(
                video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="cloud"
            )
        )
        prov = out["compute"]
        assert prov["path"] == "cloud_run"
        assert prov["requested"] == "cloud"
        assert (
            prov["execution"] == "exec-1"
        )  # the fake client's execution name (real one in prod)
        assert prov["outputs_uri"].endswith("/outputs/vidCloud/")
        assert prov["job"] and prov["region"]  # the dispatch coordinates are recorded


    def test_local_override_stamps_in_process_provenance(cloud_env):
        """On a cloud server, compute='local' runs in-process AND the result proves it:
path=in_process,
        requested=local ? so a 'ran local despite a cloud server' is auditable (no silent
ambiguity)."""
        _db, _bucket, monkeypatch = cloud_env
        _inject_fake(monkeypatch, _FakeCloudClient())
        monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)

        out = m._execute_processing(
            m.ProcessRequest(
                video_path="gs://b/clip.mp4", segmenter="motion", compute="local"
            )
        )
        assert out["status"] == "success"
        assert out["compute"] == {"path": "in_process", "requested": "local"}


    def test_default_in_process_stamps_provenance(tmp_path, monkeypatch):
        """A default-OFF (no cloud) run stamps path=in_process, requested=None ? the byte-
identical local
        path is now self-describing."""
        db = Database(str(tmp_path / "t.db"))  # noqa: F841
        monkeypatch.setattr(m, "db_instance", db)
        monkeypatch.setattr(m, "db_instance", db)
        monkeypatch.setattr(m, "global_config", AppConfig())
        m.app.state.config = AppConfig()
        monkeypatch.setattr(
            m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
        )
        monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
```

```
        video = tmp_path / "v.mp4"
        video.write_bytes(b"hashable-bytes")

        out = m._execute_processing(
            m.ProcessRequest(video_path=str(video), segmenter="motion")
        )
        assert out["compute"] == {"path": "in_process", "requested": None}


    def test_cloud_requested_unconfigured_stamps_not_dispatched(tmp_path, monkeypatch):
        """compute='cloud' on a server that can't dispatch fails clearly AND records it:
target cloud_run,
        requested=cloud, dispatched=False ? proving the cloud job never ran (no silent local
fallback)."""
        db = Database(str(tmp_path / "t.db"))  # noqa: F841
        monkeypatch.setattr(m, "db_instance", db)
        monkeypatch.setattr(m, "db_instance", db)
        monkeypatch.setattr(
            m,
            "global_config",
            AppConfig.model_validate({"deployment": {"compute_target": ""}}),
        )
        m.app.state.config = AppConfig.model_validate(
            {"deployment": {"compute_target": ""}}
        )
        monkeypatch.setattr(
            m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
        )
        monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
        monkeypatch.delenv("GOOGLE_CLOUD_PROJECT", raising=False)
        video = tmp_path / "v.mp4"
        video.write_bytes(b"hashable-bytes")

        out = m._execute_processing(
            m.ProcessRequest(
                video_path=str(video), segmenter="wasb_hybrid", compute="cloud"
            )
        )
        assert out["status"] == "failed"
        # Honest: it ran NOWHERE (pre-dispatch reject) ? path=not_run, but the intent is
recorded.
        assert out["compute"] == {
            "path": "not_run",
            "requested": "cloud",
            "target": "cloud_run",
            "dispatched": False,
        }


    def test_validation_failure_stamps_not_run_provenance(tmp_path, monkeypatch):
        """A validation failure never reaches a segmenter ? path=not_run (nothing
executed)."""
        db = Database(str(tmp_path / "t.db"))  # noqa: F841
        monkeypatch.setattr(m, "db_instance", db)
        monkeypatch.setattr(m, "global_config", AppConfig())
        m.app.state.config = AppConfig()
        fail = [
            ValidationResult(validator_name="x", passed=False, message="bad", details={})
        ]
        monkeypatch.setattr(m.registry, "run_all_with_context", lambda *a, **k: (fail, {}))
        video = tmp_path / "v.mp4"
        video.write_bytes(b"hashable-bytes")

        out = m._execute_processing(m.ProcessRequest(video_path=str(video)))
        assert out["status"] == "failed"
```

```
        assert out["compute"]["path"] == "not_run"


    def test_full_api_process_job_result_carries_cloud_provenance(cloud_env):
        """End-to-end: POST /api/process (compute=cloud) ? the job result the SPA polls
carries the
        cloud_run provenance + execution, so the UI/operator can PROVE the cloud path was
taken."""
        _db, _bucket, monkeypatch = cloud_env
        _inject_fake(monkeypatch, _FakeCloudClient(video_id="vidProv"))
        app = m.create_app(m.global_config, _db)
        client = TestClient(app)

        r = client.post(
            "/api/process",
            json={
                "video_path": "gs://b/clip.mp4",
                "segmenter": "wasb_hybrid",
                "compute": "cloud",
            },
        )
        assert r.status_code == 202
        job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
        assert job["status"] == "done"
        assert job["result"]["compute"]["path"] == "cloud_run"
        assert job["result"]["compute"]["execution"] == "exec-1"


    # --- #468: local compute lane ? CPU/GPU-heavy in-process work serializes, remote polls
don't --------


    def test_in_process_detection_runs_under_the_local_lane(cloud_env):
        """The in-process segmenter holds orchestration._LOCAL_COMPUTE_LANE while it runs,
so raising the
        drain worker count (to overlap I/O-bound remote dispatch) never runs two local
detections at once.
        A remote dispatch returns before this path, so remote polls never hold the lane
(they overlap)."""
        import backend.orchestration as orch

        _db, _bucket, monkeypatch = cloud_env
        held = {}

        class _LaneCheckSeg:
            def __init__(self, db_, cfg_):
                self.db = db_

            def process_video(self, vid, path, *a, **k):
                # the wrapping `with _LOCAL_COMPUTE_LANE` holds the size-1 lane ? a non-
blocking re-acquire
                # must FAIL while detection runs (proving concurrent local detections are
excluded).
                got = orch._LOCAL_COMPUTE_LANE.acquire(blocking=False)
                held["busy"] = not got
                if got:
                    orch._LOCAL_COMPUTE_LANE.release()
                sid = self.db.add_play_segment(vid, 0.0, 5.0)
                return [
                    {"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0,
"metadata": {}}
                ]

        monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _LaneCheckSeg)
        out = m._execute_processing(
```

```
910             m.global_config,
911             m.db_instance,
912             m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="motion",
compute="local"),
913         )
914         assert out["status"] == "success"
915         assert held.get("busy") is True  # detection ran while the local lane was held
916
917
918     def test_wait_progress_message_is_honest_and_minute_granular():
919         """#482: the CP knows ELAPSED time, not frames ? the heartbeat says so plainly,
shows
920         movement from the first tick ("starting up" during queue/cold-start), then counts
minutes."""
921         from backend.orchestration import _wait_progress_message
922
923         early = _wait_progress_message("wasb_hybrid", 12.0)
924         assert "wasb_hybrid" in early and "starting up" in early
925         later = _wait_progress_message("wasb_hybrid", 125.0)
926         assert later.endswith("2 min elapsed")
927
```

6. user (2026-07-07T14:45:24.871Z)

```
240         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
241         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
242         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
243         # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
244         compute = get_compute_backend(config)
245         if compute is not None:
246             return _dispatch_remote(compute, args)
247
248         storage_cfg = config.storage.model_dump()
249
250         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
251         os.makedirs(scratch, exist_ok=True)
252         config.output.output_dir = scratch
253
254         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
255         local_video = storage.fetch(
256             args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
257         )
258         print(f"[worker] pulled {args.video_uri} -> {local_video}")
259
260         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
261         video_id = compute_video_id(local_video)
262
263         # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
264         # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate:
it runs the
265         # SAME validators with ITS resolved config and only dispatches on PASS. Re-
validating here is
266         # redundant, decodes the whole video a SECOND time, and is outright WRONG when the
worker's config
267         # resolves different thresholds than the control-plane's ? observed 2026-07-07: a
5312x2988 clip
268         # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0,
then the worker
269         # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the
```

```
fact.
270         # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
271         # direct ``worker infer`` CLI has no control-plane, so it still validates by
default.
272         if getattr(args, "skip_validation", False):
273             print(
274                 "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
275                 "gate and already validated this input."
276             )
277             val_results, val_context = [], {}
278         else:
279             val_results, val_context = validator_registry.run_all_with_context(
280                 local_video, config
281             )
282         failures = [r for r in val_results if not r.passed]
283         db = Database(os.path.join(scratch, config.output.db_name))
284         db.add_video(
285             video_id,
286             local_video,
287             "failed" if failures else "processed",
288             [r.model_dump() for r in val_results],
289         )
290
291         def _fail(rc: int) -> int:
292             # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
293             # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
294             # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
295             # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
see an
296             # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
297             # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
298             # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
299             try:
300                 _serialize_and_push_outputs(
301                     scratch,
302                     args.out_uri,
303                     video_id,
304                     [],
305                     "failed",
306                     storage_cfg,
307                     str(getattr(args, "video_uri", "") or ""),
308                 )
309             except Exception as e:  # never mask the real failure on an artifact-push hiccup
310                 logger.warning("[worker] could not push failure artifacts: %s", e)
311             # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
312             if getattr(args, "done_uri", None):
313                 _write_done_marker(
314                     args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
315                 )
316             return rc
317
318         segments: List[dict] = []
319         segmenter_actual = None
320         segmentation_ran = False
321         status = "failed"
322         try:
323             if failures:
```

```
324                 print(
325                     "[worker] validation FAILED: "
326                     + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
327                 )
328                 return _fail(2)
329             seg_name = args.segmenter or config.indexing.default_segmenter
330             segmenter_cls = segmenter_registry.get(seg_name)
331             if not segmenter_cls:
332                 print(
333                     f"[worker] unknown segmenter: {seg_name!r} "
334                     f"(have {list(segmenter_registry.list_available())})"
335                 )
336                 return _fail(2)
337             segmenter_actual = seg_name
338             result = segmenter_cls(db, config).process_video(
339                 video_id, local_video, max_frames=args.max_frames
340             )
341             segmentation_ran = True
342             if isinstance(result, Failure):
343                 print(f"[worker] segmenter FAILED: {result.message}")
344                 return _fail(3)
345             segments = result.value if hasattr(result, "value") else result
346             status = "success"
347             # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
348             # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
349             # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
350             if not segments:
351                 detector_impl = (
352                     os.environ.get("RALLY_DETECTOR_IMPL")
353                     or getattr(config.indexing, "detector_impl", None)
354                     or "auto"
355                 )
356                 logger.warning(
357                     "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
358                     "The detector substrate yielded no usable trajectories/windows ? check
the detector "
359                     "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
```

7. assistant (2026-07-07T14:45:30.162Z)

Let me read the test_worker.py file and the rest of cmd_infer to understand the full flow.

8. user (2026-07-07T14:45:31.603Z)

```
1       """Worker dispatcher: pull (storage) ? run pipeline ? push (storage).
2
3       `cmd_infer` is exercised end-to-end over the LOCAL storage backend with the validator +
4       segmenter mocked, so it needs no real video/GPU/network. The real S3 round-trip is
validated
5       on the droplet against MinIO.
6       """
7
8       import json
9
10      import pytest
11
12      from backend.config import load_config
13      from backend.results import Failure
14      from backend.worker import __main__ as wmain
15
16      # ------------------------------------------------------------- pure helpers
```

```

    @pytest.mark.parametrize(
        "raw,expected",
        [
            ("true", True),
            ("False", False),
            ("3", 3),
            ("2.5", 2.5),
            ("wasb_hybrid", "wasb_hybrid"),
        ],
    )
    def test_coerce(raw, expected):
        assert wmain._coerce(raw) == expected and type(wmain._coerce(raw)) is type(expected)


    def test_apply_overrides_on_typed_config():
        cfg = _fresh_config()
        wmain._apply_overrides(
            cfg,
            [
                "indexing.skip_ai_handoff=true",
                "indexing.min_segment_duration=1.5",
                "sport=tennis",
            ],
        )
        assert cfg.indexing.skip_ai_handoff is True
        assert cfg.indexing.min_segment_duration == 1.5
        assert cfg.sport == "tennis"


    def test_apply_overrides_rejects_bad_format():
        cfg = _fresh_config()
        with pytest.raises(ValueError):
            wmain._apply_overrides(cfg, ["no_equals_sign"])


    def test_write_intervals_csv(tmp_path):
        segs = [
            {
                "start_time": 2.0,
                "end_time": 5.0,
                "shot_count": 4,
                "shot_count_confidence": "LocalNoAI",
            },
            {"start_time": 9.0, "end_time": 12.5},
        ]
        out = tmp_path / "intervals.csv"
        wmain._write_intervals_csv(segs, str(out))
        lines = out.read_text().splitlines()
        assert lines[0] == "start,end,shot_count,ending_reason"
        assert lines[1].startswith("2.000,5.000,4,")
        assert lines[2].startswith("9.000,12.500,,")


    def test_write_report_json(tmp_path):
        out = tmp_path / "report.json"
        wmain._write_report_json(
            "vid1", [{"start_time": 1.0, "end_time": 2.0}], "success", str(out)
        )
        data = json.loads(out.read_text())
        assert data["video_id"] == "vid1" and data["segment_count"] == 1
        assert data["segments"][0] == {"start": 1.0, "end": 2.0}


```

```
82      def test_parser_infer_accepts_set_and_uris():
83          args = wmain.build_parser().parse_args(
84              [
85                  "infer",
86                  "--video-uri",
87                  "s3://b/v.mp4",
88                  "--out-uri",
89                  "s3://b",
90                  "--set",
91                  "indexing.skip_ai_handoff=true",
92                  "--set",
93                  "sport=tennis",
94              ]
95          )
96          assert args.cmd == "infer" and args.video_uri == "s3://b/v.mp4"
97          assert args.set == ["indexing.skip_ai_handoff=true", "sport=tennis"]
98
99
100     def test_parser_infer_skip_validation_flag():
101         base = ["infer", "--video-uri", "s3://b/v.mp4", "--out-uri", "s3://b"]
102         # Default-OFF: the direct `worker infer` CLI still validates.
103         assert wmain.build_parser().parse_args(base).skip_validation is False
104         # The dispatcher passes it to disable the redundant worker re-validation.
105         assert wmain.build_parser().parse_args(base + ["--skip-validation"]).skip_validation
is True
106
107
108     # ------------------------------------------------------------- cmd_infer (local,
mocked)
109
110
111     class _OKResult:
112         passed = True
113         validator_name = "fake"
114         message = "ok"
115
116         def model_dump(self):
117             return {"validator_name": "fake", "passed": True, "message": "ok"}
118
119
120     class _FailResult(_OKResult):
```

9. user (2026-07-07T14:45:32.531Z)

```
358                     "The detector substrate yielded no usable trajectories/windows ? check
the detector "
359                     "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
360                     "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
361                     "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
362                     "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
363                     "detections, or the input resolution differs from the tuned proxy
resolution. "
364                     "Re-run with -v for the native command + frame counts.",
365                     video_id,
366                     segmenter_actual,
367                     detector_impl,
368                 )
369         finally:
370             # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
371             emit_indexer_report(
372                 db=db,
```

```
                video_id=video_id,
                video_path=local_video,
                config=config,
                val_results=val_results,
                val_context=val_context,
                segmenter_requested=args.segmenter,
                segmenter_actual=segmenter_actual,
                was_reingest=False,
                segmentation_ran=segmentation_ran,
            )

        # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
        pushed = _serialize_and_push_outputs(
            scratch,
            args.out_uri,
            video_id,
            segments,
            status,
            storage_cfg,
            str(getattr(args, "video_uri", "") or ""),
        )

        print(
            f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
        )
        for u in pushed:
            print("   ", u)

        # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
its bucket-poll
        # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
FAILURES (rc 2/3)
        # write their own marker via _fail() above, so the dispatcher fails FAST either way
(no 30-min hang
        # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
        if getattr(args, "done_uri", None):
            _write_done_marker(
                args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
            )
        return 0


    #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
    #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
    _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")

    #: Leading ISO-date prefix on a canonical label name (``labels/<YYYY-MM-
DD>_<name>.rallies.csv``,
    #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
but is NOT
    #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
    _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")


    def _label_clip_name(fname: str) -> str:
        """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label
filename.

        Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date
prefix, so a
        canonical dated label and a legacy bare label resolve to the SAME bare clip stem:
```

```
427
428         * ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094``       (canonical, date-
stamped)
429         * ``GX020094.rallies.csv``            -> ``GX020094``       (legacy bare ? same
stem)
430         * ``GX020094_proxy.rallies.csv``      -> ``GX020094_proxy`` (``_proxy`` is part of
the name, kept)
431         * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in
<name> survive)
432
433         Without the date strip, ``--trajectory-source detector`` can't match a dated label
to its bare
434         video stem, so every clip is skipped and train aborts with ``<2 videos``
(DATA_IN_GCS §7b #1).
435         """
436         name = fname.replace(".rallies.csv", "").replace(".csv", "")
437         return _LABEL_DATE_PREFIX.sub("", name)
438
439
440     def _probe_video_fps_width(path: str):
441         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
442
443         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
444         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
445         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
446         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
447         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
448         rather than guesses.
449         """
450         import json
451         import subprocess
452
453         try:
454             out = subprocess.check_output(
455                 [
456                     ffprobe_bin(),
457                     "-v",
458                     "error",
459                     "-select_streams",
460                     "v:0",
461                     "-show_entries",
462                     "stream=r_frame_rate,width",
463                     "-of",
464                     "json",
465                     path,
466                 ],
467                 stderr=subprocess.DEVNULL,
468             ).decode()
469             st = (json.loads(out).get("streams") or [{}])[0]
470             num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
471             fps = float(num) / float(den or "1")
472             width = float(st.get("width") or 0)
473             if fps > 0 and width > 0:
474                 return fps, width
475         except (
476             subprocess.SubprocessError,
477             ValueError,
```

10. assistant (2026-07-07T14:45:39.790Z)

Let me read the rest of the test_worker.py around the cmd_infer tests and helpers.

11. user (2026-07-07T14:45:41.534Z)

```
120     class _FailResult(_OKResult):
121         passed = False
122         message = "too blurry"
123 
124 
125     class _FakeSeg:
126         def __init__(self, db, config):
127             pass
128 
129         def process_video(self, video_id, path, max_frames=None):
130             return [
131                 {
132                     "start_time": 2.0,
133                     "end_time": 5.0,
134                     "shot_count": 4,
135                     "shot_count_confidence": "LocalNoAI",
136                 },
137                 {
138                     "start_time": 9.0,
139                     "end_time": 12.0,
140                     "shot_count": 6,
141                     "shot_count_confidence": "LocalNoAI",
142                 },
143             ]
144 
145 
146     def _fresh_config():
147         # Load defaults without touching any cwd config.json.
148         import tempfile
149 
150         p = tempfile.NamedTemporaryFile("w", suffix=".json", delete=False)
151         p.write("{}")
152         p.close()
153         return load_config(p.name)
154 
155 
156     @pytest.fixture
157     def mocked_pipeline(monkeypatch):
158         monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vid123")
159         monkeypatch.setattr(
160             wmain.validator_registry,
161             "run_all_with_context",
162             lambda path, cfg: ([_OKResult()], {}),
163         )
164         monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FakeSeg)
165         monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
166 
167 
168     def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):
169         video = tmp_path / "in.mp4"
170         video.write_bytes(b"FAKEVIDEO")
171         cfg = tmp_path / "config.json"
172         cfg.write_text("{}")
173         out = tmp_path / "bucket"
174         scratch = tmp_path / "scratch"
175 
176         rc = wmain.main(
177             [
178                 "infer",
179                 "--video-uri",
180                 str(video),
```

```
                "--out-uri",
                str(out),
                "--config",
                str(cfg),
                "--scratch",
                str(scratch),
                "--segmenter",
                "wasb_hybrid",
                "--set",
                "indexing.skip_ai_handoff=true",
            ]
        )
        assert rc == 0
        intervals = out / "outputs" / "vid123" / "intervals.csv"
        report = out / "outputs" / "vid123" / "report.json"
        assert intervals.exists() and report.exists()
        rows = intervals.read_text().splitlines()
        assert len(rows) == 3  # header + 2 rallies
        assert json.loads(report.read_text())["segment_count"] == 2


    class _EmptySeg:
        """A segmenter that finds nothing ? a LEGITIMATE 0-rally match (a bare [])."""

        def __init__(self, db, config):
            pass

        def process_video(self, video_id, path, max_frames=None):
            return []


    class _FailSeg:
        """A segmenter whose DETECTOR failed/timed out ? returns a Failure (e.g. WASB
produced no
        trajectory), which is NOT the same as a 0-rally match. This is the wf294 shape."""

        def __init__(self, db, config):
            pass

        def process_video(self, video_id, path, max_frames=None):
            return Failure(
                message="WASB detector produced no trajectory (timed out or aborted)",
                is_recoverable=True,
            )


    def test_setup_logging_levels():
        import logging

        wmain._setup_logging(False)
        assert logging.getLogger().level == logging.INFO
        wmain._setup_logging(True)
        assert logging.getLogger().level == logging.DEBUG


    def test_cmd_infer_zero_segments_is_logged_loudly(tmp_path, monkeypatch, caplog):
        # The previous silent "0 segment(s)" gave nothing to analyse (the real cold-start
bug).
        monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidZ")
        monkeypatch.setattr(
            wmain.validator_registry,
            "run_all_with_context",
            lambda path, cfg: ([_OKResult()], {}),
        )
        monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _EmptySeg)
```

```
        monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
        monkeypatch.setenv("RALLY_DETECTOR_IMPL", "native")
        video = tmp_path / "in.mp4"
        video.write_bytes(b"FAKE")
        out = tmp_path / "bucket"

        # Call cmd_infer directly (not via main(), whose _setup_logging force-reconfigures
handlers and
        # would detach caplog's). The warning is emitted by cmd_infer regardless of who
configures logging.
        args = wmain.build_parser().parse_args(
            [
                "infer",
                "--video-uri",
                str(video),
                "--out-uri",
                str(out),
                "--scratch",
                str(tmp_path / "s"),
                "--segmenter",
                "wasb_hybrid",
            ]
        )
        with caplog.at_level("WARNING", logger="backend.worker"):
            rc = wmain.cmd_infer(args)
        assert rc == 0  # still a clean run, just empty ? but now EXPLAINED
        msgs = " ".join(r.getMessage() for r in caplog.records)
        assert "0 segments" in msgs and "vidZ" in msgs
        assert "detector_impl=native" in msgs  # surfaces the resolved detector
        assert "native runner UNAVAILABLE" in msgs  # points at the likely cause
        # A segmenter that returns a bare [] is a LEGITIMATE 0-rally match ? status stays
"success"
        # (contrast test_cmd_infer_detector_failure_*, where a Failure surfaces as
status="failed").
        report = json.loads((out / "outputs" / "vidZ" / "report.json").read_text())
        assert report["segment_count"] == 0 and report["status"] == "success"


    def test_cmd_infer_detector_failure_marks_report_and_marker_failed(
        tmp_path, monkeypatch
    ):
        """wf294 fix: a segmenter that returns a Failure (detector timeout/abort) must NOT
be written
        as a 0-segment "success". cmd_infer returns rc 3 and writes report.json + the done-
marker with
        status="failed" so the control plane / alpha-user surface can show an error /
retry."""
        monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidX")
        monkeypatch.setattr(
            wmain.validator_registry,
            "run_all_with_context",
            lambda path, cfg: ([_OKResult()], {}),
        )
        monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FailSeg)
        monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
        video = tmp_path / "in.mp4"
        video.write_bytes(b"FAKE")
        out = tmp_path / "bucket"
        done = tmp_path / "bucket" / "_dispatch" / "tok.done"

        rc = wmain.main(
            [
                "infer",
                "--video-uri",
                str(video),
```

```
                "--out-uri",
                str(out),
                "--scratch",
                str(tmp_path / "s"),
                "--segmenter",
                "wasb_hybrid",
                "--done-uri",
                str(done),
            ]
        )
        # A detector failure is rc 3 (distinct from validation's rc 2 and a clean rc 0).
        assert rc == 3
        # report.json is written as an explicit FAILURE, not a false empty "success".
        report = json.loads((out / "outputs" / "vidX" / "report.json").read_text())
        assert report["status"] == "failed" and report["segment_count"] == 0
        assert report["segments"] == []
        # ...and the completion marker the cloud dispatcher bucket-polls carries the same
verdict.
        marker = json.loads(done.read_text())
        assert marker["returncode"] == 3 and marker["status"] == "failed"
        assert marker["segment_count"] == 0 and marker["video_id"] == "vidX"


    def test_cmd_infer_real_trajectory_segmenter_failure_flows_to_report(tmp_path,
monkeypatch):
        """End-to-end: the ACTUAL WasbHybridSegmenter no-trajectory abort (the wf294 shape)
flows
        through cmd_infer -> _fail(3) -> a status="failed" report.json + done-marker. Locks
the two
        halves (segmenter Failure + worker report/marker) TOGETHER, not just via the
_FailSeg stub."""
        from unittest.mock import MagicMock

        from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter

        runner = MagicMock()
        runner.healthcheck.return_value = (True, "env ok")
        runner.run_predict.return_value = None  # WASB timed out / aborted: no trajectory
produced

        # Force the real base process_video down to its no-trajectory abort without a
GPU/video.
        monkeypatch.setattr(WasbHybridSegmenter, "_resolve_runner", lambda self, cfg:
(runner, {}))
        monkeypatch.setattr(
            WasbHybridSegmenter, "_probe_fps_width", lambda *a, **k: (30.0, 1280.0)
        )
        monkeypatch.setattr(
            "backend.pipeline.segmenters.trajectory_hybrid.get_video_duration", lambda p:
30.0
        )
        monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidR")
        monkeypatch.setattr(
            wmain.validator_registry, "run_all_with_context", lambda path, cfg:
([_OKResult()], {})
        )
        monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name:
WasbHybridSegmenter)
        monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)

        video = tmp_path / "in.mp4"
        video.write_bytes(b"FAKE")
        out = tmp_path / "bucket"
        done = tmp_path / "bucket" / "_dispatch" / "tok.done"

```

```
        rc = wmain.main(
            [
                "infer",
                "--video-uri",
                str(video),
                "--out-uri",
                str(out),
                "--scratch",
                str(tmp_path / "s"),
                "--segmenter",
                "wasb_hybrid",
                "--done-uri",
                str(done),
                "--set",
                "indexing.skip_ai_handoff=true",  # no API key needed; isolate the detector
abort
            ]
        )
        assert rc == 3
        report = json.loads((out / "outputs" / "vidR" / "report.json").read_text())
        assert report["status"] == "failed" and report["segment_count"] == 0
        marker = json.loads(done.read_text())
        assert marker["returncode"] == 3 and marker["status"] == "failed"


    def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
        monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidF")
        monkeypatch.setattr(
            wmain.validator_registry,
            "run_all_with_context",
            lambda path, cfg: ([_FailResult()], {}),
        )
        monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
        video = tmp_path / "in.mp4"
        video.write_bytes(b"X")
        cfg = tmp_path / "config.json"
        cfg.write_text("{}")
        out = tmp_path / "o"
        done = tmp_path / "o" / "_dispatch" / "tok.done"
        rc = wmain.main(
            [
                "infer",
                "--video-uri",
                str(video),
                "--out-uri",
                str(out),
                "--config",
                str(cfg),
                "--scratch",
                str(tmp_path / "s"),
                "--done-uri",
                str(done),
            ]
        )
        assert rc == 2
        # A validation failure is ALSO written as an explicit failure (not a missing
artifact / false
        # success): report.json status="failed" + the done-marker rc 2 / status="failed".
        report = json.loads((out / "outputs" / "vidF" / "report.json").read_text())
        assert report["status"] == "failed" and report["segment_count"] == 0
        marker = json.loads(done.read_text())
        assert marker["returncode"] == 2 and marker["status"] == "failed"


    def test_cmd_infer_skip_validation_bypasses_validators(tmp_path, monkeypatch):
```

```
419         """--skip-validation (set by the dispatcher, the authoritative gate) must NOT run
the worker's
420         validators ? no second decode, and no chance of a threshold mismatch REJECTING a
clip the
421         control-plane already passed (the live 2026-07-07 CUJ failure). Prove it by making
422         run_all_with_context RAISE: a skipped run never touches it, so the job completes rc
0 with a
423         status="success" report. Contrast test_cmd_infer_validation_failure_returns_2 (no
flag ? rc 2)."""
424         monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidSkip")
425
426         def _boom(*a, **k):  # the validators must never be reached on the skip path
427             raise AssertionError("run_all_with_context must not run under --skip-
validation")
428
429         monkeypatch.setattr(wmain.validator_registry, "run_all_with_context", _boom)
430         monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FakeSeg)
431         monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
432         video = tmp_path / "in.mp4"
433         video.write_bytes(b"FAKE")
434         out = tmp_path / "bucket"
435
436         rc = wmain.main(
437             [
438                 "infer",
439                 "--video-uri",
440                 str(video),
441                 "--out-uri",
442                 str(out),
443                 "--scratch",
444                 str(tmp_path / "s"),
445                 "--segmenter",
446                 "wasb_hybrid",
447                 "--skip-validation",
448             ]
449         )
450         assert rc == 0  # a clip that WOULD fail re-validation still succeeds ? the gate is
trusted
451         report = json.loads((out / "outputs" / "vidSkip" / "report.json").read_text())
452         assert report["status"] == "success" and report["segment_count"] == 2
453
454
455     # --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push bundle
---
456
457
458     @pytest.mark.parametrize(
459         "fname,expected",
```

12. assistant (2026-07-07T14:46:29.802Z)

Let me check for tests of `_dispatch_remote` propagation and direct cloud_run argv building.

13. user (2026-07-07T14:46:31.964Z)

```
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\backend\worker\__main__.py:174:def
_dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\backend\worker\__main__.py:189:
skip_validation=bool(getattr(args, "skip_validation", False)),
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\backend\worker\__main__.py:246:
return _dispatch_remote(compute, args)
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\backend\worker\__main__.py:270:    #
--skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote) skips it; the
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\backend\worker\__main__.py:272:    if
getattr(args, "skip_validation", False):
```

```
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\backend\worker\__main__.py:274:
"[worker] validation SKIPPED (--skip-validation): the dispatcher is the authoritative "
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\backend\worker\__main__.py:744:
"--skip-validation",
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\tests\test_api_cloud_dispatch.py:128:
"""Make backend.compute.get_compute_backend (re-imported inside _maybe_dispatch_remote) build a
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\tests\test_api_cloud_dispatch.py:245:
carry --skip-validation, telling the worker to trust the gate. Regression guard for that
wiring."""
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\tests\test_api_cloud_dispatch.py:252:
assert "--skip-validation" in fake.calls[0]
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-
reval\docs\COMPUTE_DECOUPLED_SERVING\runlogs\DEPLOY_REPORT_cloud-
serving_a1-cuj_2026-07-04.md:53:- **WASB batch tuning is NOT forwarded on the dispatch path.**
`_maybe_dispatch_remote` forwards only
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-
reval\docs\COMPUTE_DECOUPLED_SERVING\runlogs\DEPLOY_REPORT_cloud-
serving_a1-cuj_2026-07-04.md:67:   `/tmp/apphome/output/uploads/?`), and
`_maybe_dispatch_remote` hard-rejects local input under
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\tests\test_compute_backend.py:413:
def run_infer(self, spec):
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-
reval\docs\archives\past_projects\AUDIT_REPORT_khelsutra-
guru_2026-07-02-COMPLETED-2026-07-02.md:73:[Omitted long matching line]
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\backend\main.py:241:
_maybe_dispatch_remote,
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\backend\main.py:268:    # default
(deployment.compute_target). Only honoured for these two values; see _maybe_dispatch_remote.
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\backend\orchestration.py:4:Cloud Run
dispatch + ingest path (``_maybe_dispatch_remote`` / ``_ingest_remote_segments``),
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\backend\orchestration.py:391:def
_maybe_dispatch_remote(
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\backend\orchestration.py:582:
skip_validation=True,
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\backend\orchestration.py:728:    #
_maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config fail).
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\backend\orchestration.py:805:
remote = _maybe_dispatch_remote(
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\tests\test_main_module_budget.py:48:
"_maybe_dispatch_remote",
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\backend\compute\cloud_run.py:179:
def run_infer(
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\backend\compute\cloud_run.py:199:
if spec.skip_validation:
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\backend\compute\cloud_run.py:202:
argv.append("--skip-validation")
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\backend\compute\base.py:37:
skip_validation: bool = False
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\backend\compute\base.py:63:    def
run_infer(
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\backend\api\job_worker.py:137:
# v8 compute-picker: the worker runs _execute_processing ? _maybe_dispatch_remote, which
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\backend\api\job_worker.py:364:
# Same normalization the execution path applies (_maybe_dispatch_remote lowercases
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\tests\test_worker.py:100:def
test_parser_infer_skip_validation_flag():
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\tests\test_worker.py:103:    assert
wmain.build_parser().parse_args(base).skip_validation is False
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\tests\test_worker.py:105:    assert
wmain.build_parser().parse_args(base + ["--skip-validation"]).skip_validation is True
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\tests\test_worker.py:418:def
test_cmd_infer_skip_validation_bypasses_validators(tmp_path, monkeypatch):
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\tests\test_worker.py:419:    """--
skip-validation (set by the dispatcher, the authoritative gate) must NOT run the worker's
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\tests\test_worker.py:427:
```

```
raise AssertionError("run_all_with_context must not run under --skip-validation")
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\tests\test_worker.py:447:
"--skip-validation",
```

14. user (2026-07-07T14:46:32.886Z)

```
160             policy: ExecutionPolicy = DEFAULT_POLICY,
161             token: Optional[str] = None,
162             container_overrides: Optional[tuple[str, ...]] = None,
163         ) -> None:
164             self._client = run_client
165             self._job_name = job_name
166             self._region = region
167             self._project = project
168             self._storage_config = storage_config or {}
169             self._policy = policy
170             # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
171             # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
172             self._token = token
173             self._container_overrides = (
174                 _DEFAULT_CONTAINER_OVERRIDES
175                 if container_overrides is None
176                 else tuple(container_overrides)
177             )
178
179         def run_infer(
180             self,
181             spec: ComputeJobSpec,
182             *,
183             on_wait: "Callable[[float], None] | None" = None,
184         ) -> ComputeResult:
185             out_root = spec.out_uri.rstrip("/")
186             token = self._token or uuid.uuid4().hex
187             done_uri = f"{out_root}/_dispatch/{token}.done"
188             argv: List[str] = [
189                 "infer",
190                 "--video-uri",
191                 spec.video_uri,
192                 "--out-uri",
193                 spec.out_uri,
194                 "--done-uri",
195                 done_uri,
196             ]
197             if spec.segmenter:
198                 argv += ["--segmenter", spec.segmenter]
199             if spec.skip_validation:
200                 # The control-plane already validated + only dispatches on PASS ? tell the
worker not to
201                 # re-decode and re-validate (redundant, and its config may resolve different
thresholds).
202                 argv.append("--skip-validation")
203             for kv in (*spec.config_overrides, *self._container_overrides):
204                 argv += ["--set", kv]
205
206             execution = self._client.run_job(
207                 self._job_name, argv, region=self._region, project=self._project
208             )
209             LOG.info(
210                 "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
211                 self._job_name,
212                 execution,
213                 done_uri,
214             )
```

```
215
216             try:
217                 marker = poll_until(
218                     lambda: self._read_marker(done_uri),
219                     self._policy,
220                     label="cloud-run-infer",
221                     logger=LOG,
222                     # Live heartbeat (#482): each still-waiting tick reaches the caller so
223                     # job.progress moves during the whole GPU run instead of freezing.
224                     on_tick=on_wait,
225                 )
226             except PolicyTimeout as timeout_exc:
227                 marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
228             video_id = str(marker.get("video_id", ""))
229             # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
```

15. assistant (2026-07-07T14:46:37.575Z)

Let me check test_compute_backend.py for direct cloud_run argv-building tests and any `_dispatch_remote` test.

16. user (2026-07-07T14:46:40.359Z)

```
15:from backend.compute import ComputeJobSpec, get_compute_backend
16:from backend.compute.cloud_run import CloudRunCompute, CloudRunJobClient
26:    exactly where the real worker would, so CloudRunCompute's bucket-poll resolves
immediately."""
36:    def run_job(self, job_name, argv, *, region, project=None):
38:            {"job": job_name, "argv": list(argv), "region": region, "project": project}
40:        out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
41:        done_uri = argv[argv.index("--done-uri") + 1]
80:    def run_job(self, job_name, argv, *, region, project=None):
82:            {"job": job_name, "argv": list(argv), "region": region, "project": project}
101:    def run_job(self, job_name, argv, *, region, project=None):
103:            {"job": job_name, "argv": list(argv), "region": region, "project": project}
125:def test_factory_is_none_for_local_targets():
132:def test_factory_returns_cloud_run_for_cloud_target():
134:    assert isinstance(backend, CloudRunCompute)
140:def test_real_client_rejects_missing_project():
155:def test_real_client_cancel_rejects_unresolvable_execution_name():
168:# ------------------------------------------------------------ CloudRunCompute dispatch +
poll
171:def test_cloud_run_dispatch_polls_and_returns_result(tmp_path):
173:    backend = CloudRunCompute(
183:    result = backend.run_infer(
184:        ComputeJobSpec(
201:def test_dispatch_argv_forces_inline_and_carries_overrides(tmp_path):
205:    backend = CloudRunCompute(
208:    backend.run_infer(
209:        ComputeJobSpec(
217:    argv = client.calls[0]["argv"]
218:    assert argv[0] == "infer"
219:    assert "--done-uri" in argv and "--video-uri" in argv and "--out-uri" in argv
220:    assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
222:    joined = " ".join(argv)
226:    # user override precedes the inline-forcing overrides in --set order
232:def test_unique_token_per_call_when_not_injected(tmp_path):
235:    backend = CloudRunCompute(run_client=client, job_name="j", region="r", policy=_FAST)
236:    backend.run_infer(
237:        ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk"))
239:    backend.run_infer(
240:        ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk"))
242:    done1 = client.calls[0]["argv"][client.calls[0]["argv"].index("--done-uri") + 1]
243:    done2 = client.calls[1]["argv"][client.calls[1]["argv"].index("--done-uri") + 1]
```

```
247:def test_read_json_passes_dst_for_cloud_backends(monkeypatch):
266:    backend = cr.CloudRunCompute(run_client=_FakeRunClient(), job_name="j", region="r")
275:def test_cloud_run_polls_until_marker_appears(monkeypatch):
298:    backend = cr.CloudRunCompute(
307:    result = backend.run_infer(
308:        ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri="gs://b")
322:def test_poll_timeout_cancels_execution_and_raises(tmp_path):
326:    backend = CloudRunCompute(
334:        backend.run_infer(
335:            ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
344:def test_poll_timeout_cancel_failure_still_raises_policy_timeout(tmp_path):
348:    backend = CloudRunCompute(
356:        backend.run_infer(
357:            ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
366:def test_poll_timeout_late_marker_rescues_result(monkeypatch):
390:    backend = cr.CloudRunCompute(
393:    result = backend.run_infer(
394:        ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri="gs://b")
405:def test_cmd_infer_dispatches_when_cloud_target(tmp_path, monkeypatch):
413:        def run_infer(self, spec):
446:def test_worker_writes_done_marker_on_success(tmp_path, monkeypatch):
515:def test_worker_no_done_uri_is_unchanged(tmp_path, monkeypatch):
570:def test_worker_writes_failure_marker_on_validation_fail(tmp_path, monkeypatch):
630:def test_worker_failure_writes_no_marker_without_done_uri(tmp_path, monkeypatch):
684:def test_dockerfile_has_required_directives():
714:def test_dockerfile_uses_micromamba_not_anaconda_tos():
730:def test_remote_poll_budget_comes_from_config_unless_policy_injected():
754:def test_run_infer_forwards_on_wait_into_the_bucket_poll(tmp_path, monkeypatch):
755:    """The #482 heartbeat plumbing: the on_wait callback handed to run_infer must be the
767:    backend = CloudRunCompute(
779:    result = backend.run_infer(
780:        ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk")),
```

17. user (2026-07-07T14:46:46.894Z)

```
201     def test_dispatch_argv_forces_inline_and_carries_overrides(tmp_path):
202         """The dispatched container must run INLINE (never re-dispatch) ?
compute_target=local_gpu +
203         native detector appended AFTER the user overrides (so they win); --done-uri always
present."""
204         client = _FakeRunClient()
205         backend = CloudRunCompute(
206             run_client=client, job_name="j", region="r", policy=_FAST, token="tok"
207         )
208         backend.run_infer(
209             ComputeJobSpec(
210                 video_uri="gs://b/v.mp4",
211                 out_uri=str(tmp_path / "bk"),
212                 segmenter="wasb_hybrid",
213                 config_overrides=("sport=tennis",),
214             )
215         )
216
217         argv = client.calls[0]["argv"]
218         assert argv[0] == "infer"
219         assert "--done-uri" in argv and "--video-uri" in argv and "--out-uri" in argv
220         assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
221         # the container-inline overrides come last (win) and the user override is carried
through
222         joined = " ".join(argv)
223         assert "deployment.compute_target=local_gpu" in joined
224         assert "indexing.detector_impl=native" in joined
225         assert "sport=tennis" in joined
226         # user override precedes the inline-forcing overrides in --set order
227         assert joined.index("sport=tennis") < joined.index(
```

```
228             "deployment.compute_target=local_gpu"
229         )
230
231
232     def test_unique_token_per_call_when_not_injected(tmp_path):
233         """No injected token ? a fresh uuid per dispatch, so concurrent jobs don't collide
on the marker."""
234         client = _FakeRunClient()
235         backend = CloudRunCompute(run_client=client, job_name="j", region="r", policy=_FAST)
236         backend.run_infer(
237             ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk"))
238         )
239         backend.run_infer(
240             ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk"))
241         )
242         done1 = client.calls[0]["argv"][client.calls[0]["argv"].index("--done-uri") + 1]
243         done2 = client.calls[1]["argv"][client.calls[1]["argv"].index("--done-uri") + 1]
244         assert done1 != done2
245
```

18. user (2026-07-07T14:46:47.516Z)

```
405     def test_cmd_infer_dispatches_when_cloud_target(tmp_path, monkeypatch):
406         """cmd_infer with compute_target=cloud_run hands off to the ComputeBackend (no in-
process run)."""
407         from backend.compute.base import ComputeResult
408         from backend.worker import __main__ as wmain
409
410         seen = {}
411
412         class _FakeBackend:
413             def run_infer(self, spec):
414                 seen["spec"] = spec
415                 return ComputeResult(
416                     returncode=0, video_id="vidX", outputs_uri="gs://b/outputs/vidX/"
417                 )
418
419         monkeypatch.setattr(wmain, "get_compute_backend", lambda config: _FakeBackend())
420         # cloud-serving startup checks require the AI-handoff key (AI_HANDOFF_KEY_MISSING is
fatal).
421         monkeypatch.setenv("GEMINI_API_KEY", "test-key")
422         cfg = tmp_path / "c.json"
423         cfg.write_text(
424             '{"deployment": {"profile": "cloud-serving"}, "storage": {"backend": "gcs"}}'
425         )
426         rc = wmain.cmd_infer(
427             wmain.build_parser().parse_args(
428                 [
429                     "infer",
430                     "--video-uri",
431                     "gs://b/videos/clip.mp4",
432                     "--out-uri",
433                     "gs://b",
434                     "--config",
435                     str(cfg),
436                     "--segmenter",
437                     "wasb_hybrid",
438                 ]
439             )
440         )
441         assert rc == 0
442         assert seen["spec"].video_uri == "gs://b/videos/clip.mp4"
443         assert seen["spec"].segmenter == "wasb_hybrid"
444
445
```

```
446     def test_worker_writes_done_marker_on_success(tmp_path, monkeypatch):
447         """The container side: cmd_infer with --done-uri writes a completion marker
(video_id + rc) so a
448         remote dispatcher's bucket-poll terminates. Driven by the deterministic stub + skip-
AI."""
449         from backend.worker import __main__ as wmain
```

19. assistant (2026-07-07T14:47:30.581Z)

The tests mock `emit_indexer_report`, but on the skip path it's called with `val_results=[], val_context={}`. Let me verify the real function tolerates empty validation inputs, since a crash there would be invisible to these tests.

20. user (2026-07-07T14:47:32.283Z)

Found 1 file
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\backend\reporting__init__.py

21. user (2026-07-07T14:47:38.831Z)

```
29:def emit_indexer_report(
30-    *,
31-    db,
32-    video_id: str,
--
35:    val_results: List[Any],
36:    val_context: Optional[Dict[str, Any]] = None,
37-    segmenter_requested: Optional[str] = None,
38-    segmenter_actual: Optional[str] = None,
39-    was_reingest: bool = False,
--
64:            val_results=val_results,
65:            val_context=val_context,
66-            play_segments=play_segments,
67-            candidates=candidates,
68-            segmenter_requested=segmenter_requested,
```

22. user (2026-07-07T14:47:43.937Z)

```
29      def emit_indexer_report(
30          *,
31          db,
32          video_id: str,
33          video_path: Optional[str],
34          config: Optional[Dict[str, Any]],
35          val_results: List[Any],
36          val_context: Optional[Dict[str, Any]] = None,
37          segmenter_requested: Optional[str] = None,
38          segmenter_actual: Optional[str] = None,
39          was_reingest: bool = False,
40          segmentation_ran: bool = True,
41      ) -> Optional[Dict[str, str]]:
42          """Harvest + write the report. Returns the written file paths, or None if
43          reporting is unavailable/disabled/failed. Never raises."""
44          config = config or {}
45          # Accept either a plain dict or a typed config object (e.g.
backend.config.AppConfig).
46          # Normalize to a plain nested dict so harvest's dict access works uniformly.
47          if hasattr(config, "model_dump"):
48              try:
49                  config = config.model_dump(mode="json")
50              except Exception:
51                  config = {}
52          # Consume run signals unconditionally (even when reporting is off) so the
```

```
53          # process-local store never accumulates entries for finished runs.
54          signals = run_signals.pop(video_id)
55          if not OBSREPORT_AVAILABLE or not report_enabled(config):
56              return None
57          try:
58              play_segments = db.query_play_segments(video_id, {}) if db is not None else []
59              candidates = db.query_candidate_segments(video_id) if db is not None else []
60              rec = harvest.record_run(
61                  video_id=video_id,
62                  video_path=video_path,
63                  config=config,
64                  val_results=val_results,
65                  val_context=val_context,
66                  play_segments=play_segments,
67                  candidates=candidates,
68                  segmenter_requested=segmenter_requested,
69                  segmenter_actual=segmenter_actual,
70                  was_reingest=was_reingest,
71                  segmentation_ran=segmentation_ran,
72                  run_signals=signals,
73              )
74              output_dir = (config.get("report") or {}).get("output_dir")
75              res = rec.write(output_dir=output_dir)
76              return res.paths or None
77          except Exception:  # pragma: no cover - defensive; reporting must never break a run
78              return None
79
```

23. assistant (2026-07-07T14:48:34.222Z)

I now have a complete picture. Let me verify one more thing: whether any test locks the "direct CLI validates by default" property at the cmd_infer level (it does ? `test_cmd_infer_validation_failure_returns_2` runs without the flag and validators execute ? rc 2), and confirm `emit_indexer_report` never raises on the empty `val_context`/`val_results` the skip path passes (it's wrapped in a catch-all that returns None, so no hidden crash there).

My analysis is complete. Here is what I found.

What the tests DO prove (solid):
- `test_parser_infer_skip_validation_flag` locks the parser: default `False`, flag ? `True`.
- `test_cmd_infer_skip_validation_bypasses_validators` proves the worker honors the flag end-to-end ? it makes `run_all_with_context` *raise*, so a clip that would fail (or even error) re-validation still finishes rc 0 / status="success". This directly proves the acceptance criterion at the worker level.
- `test_cloud_dispatch_skips_worker_revalidation` proves the control-plane emits `--skip-validation` into the worker argv.
- The "direct CLI still validates" negative path is covered by the pre-existing `test_cmd_infer_validation_failure_returns_2` (no flag ? validators run ? rc 2).

The chain (dispatcher emits flag ? parser parses identical literal ? cmd_infer honors it) is covered across three tests sharing the exact `--skip-validation` literal, so a typo would be caught. That is adequate.

The real gaps: the *default-off at the argv builder* and the *`_dispatch_remote` propagation* are both unguarded.

24. user (2026-07-07T14:48:49.947Z)

Structured output provided successfully